HARNESS LAB · MÓDULO 4 · AULA 4.2

O teste verde *que mente*

Verde só prova o caminho que o teste percorreu. Se ele nunca passa por onde a feature fala com fora, ele não sabe se ela funciona.

`structured output` `strict` `required` `->nullable()` `Http::fake()` `->group('integration')` `PreToolUse` `exit 2` `.claude/aceite.txt`

VOCÊ SAI COM

Um teste de contrato no ponto em que a sua feature-alvo fala com fora, já visto vermelho pelo motivo certo. A chamada real num grupo fora do comando do juiz. E o `guarda-aceite.py` impedindo o agente de reescrever esse teste para passar.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula abre com uma tese: **nada ainda supera a boa e velha navegação na aplicação**. O caso é real, de um evento, o AI Engineering Lab, em maio de 2026: o **Worthly** responde se vale a pena comprar um produto, com uma API em Laravel que chama a OpenAI com structured output. O processo do curso rodou à noite e fechou 8 fases em cerca de 1 hora (18:05 a 19:07), uma por commit.

162/162 testes passando no log da fase 8, relatados pelo agente (a suíte não foi rodada de novo): pelo critério do loop, pronto. Na manhã seguinte, a primeira requisição real à OpenAI voltou **HTTP 400**, "Invalid schema for response_format". Nenhum dos 162 passava por ali.

Por que tudo passava

A spec mandou trocar a IA por um dublê (o mock da aula), que recebia o schema e não fazia nada com ele. Os 162 provaram o app com o dublê, não com a OpenAI.

Nullable não é opcional

No strict fixo do `laravel/ai` 0.6.8, o `->nullable()` sem `->required()` ficou fora do `required`. OpenAI: "all fields or function parameters must be specified as required."

Um **end-to-end navegando com a integração real** teria pegado, até durante o processo. A cada edição sairia caro, então fica um **teste completo no fim**: feature inteira, integrações reais, dados de staging. "Os testes valem para 99% dos casos, mas aquele 1% sempre vai existir", e só a navegação pega; a automação dela, com Playwright, é a aula 4.4.

Atualizou o pacote, atualize o teste

Pacote desatualizado, provedor mudando, ferramenta que se altera. Atualizar um pacote inclui o dublê e os testes na mesma tarefa, prevista na especificação.

E não parou aí

Coluna do preço: `varchar(100)`. O dublê devolvia `'$80 - $110'`; o modelo escreve mais. Dublê mais permissivo que o provedor dá teste verde sobre cenário impossível.

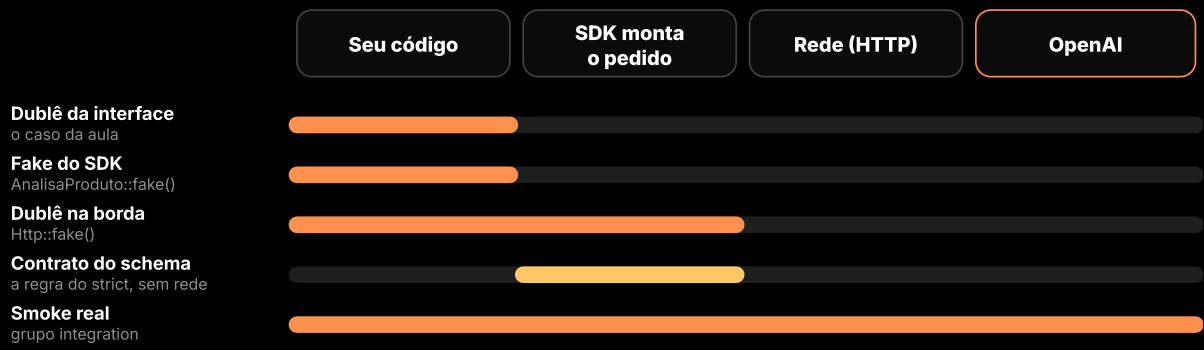
Para a aula, quem antecipa esse erro é a especificação (passo 12). E cada teste passa por **quatro perguntas**: confere o **contrato do que sai**, pelo caminho do SDK? O **dublê respeita o provedor** (texto longo, campo nulo, o erro real)? Existe um **smoke real, separado**? Ele **já foi vermelho uma vez**, pelo motivo certo? O teste que faltou é o **contrato do schema, sem rede**: em Pest, monta o schema pelo caminho do SDK e falha com o nome de cada campo fora do `required`; em qualquer stack, o `schema_estrito.py`.

A **suíte padrão**, sem rede (dublê e contrato), é o juiz e roda a cada ciclo; o **smoke real**, uma chamada com chave, roda com você antes do PR, num grupo separado. A primeira chamada a aula põe **enquanto especifica**: o smoke mostra os campos obrigatórios e dá insumo para o dublê e o juiz.

Por fim, **o agente também mexe no teste**: muda o teste para bater com o código ("modifying the problematic tests themselves to match the code's output", Claude 3.7 Sonnet System Card, p. 22) ou apaga o que falha. Pedir no prompt ajuda; obrigar é hook, igual à 2.3: o `guarda-aceite.py` barra (exit 2) toda mudança nos testes do `aceite.txt`, menos tirar o marcador de pendente.

2 Indo *além da aula*

Até onde cada tipo de teste vai: a barra é o caminho que ele percorre de verdade.



ALÉM DA AULA · O DUBLÊ CERTO FICA NA BORDA, COM A FORMA REAL

Trocar o dublê escrito à mão pelo fake do SDK não resolve: no `laravel/ai`, `AnalisaProduto::fake([...])` devolve a resposta pronta sem montar o pedido, e com o agente quebrado do caso o teste fica verde. O que pega é o dublê na **borda HTTP**: `Http::fake()` troca só a rede, e o SDK monta o pedido de verdade (com um JSON Schema cru no agente, estourou o mesmo `TypeError` que travou o caso antes do 400). O smoke dá a **forma real**: grava a resposta crua em `tests/Fixtures/openai/gravada/`. O texto longo e o `null`, uma chamada real não garante; você os injeta sobre essa forma, na **borda** (texto maior que a coluna, `null` em cada campo nullable). O smoke só sobrescreve a gravada. O `required` o dublê não confere: isso é o contrato do schema.

Testado no `laravel/ai` 1.3.0 com Pest 5.3.5: o fake usa o `FakeTextGateway`, que não passa pelo `buildSchemaFormat`. Com um servidor falso local (`OPENAI_URL`) devolvendo outra resposta, o smoke regravou a gravada e o juiz seguiu verde.

ALÉM DA AULA · O BANCO DO TESTE TAMBÉM É DUBLÊ

O `phpunit.xml` do starter kit do Laravel roda a suíte em SQLite em memória, e o SQLite ignora o tamanho do `VARCHAR`: numa coluna `string('faixa_de_preco', 100)`, ele gravou 153 caracteres sem reclamar. O Postgres do caso recusou com `SQLSTATE[22001]: value too long`. Se a feature grava texto que vem de fora, rode o juiz no mesmo banco de produção (o Sail sobe um), valide o tamanho (`max:100`) ou use `text`.

sqlite.org, Datatypes In SQLite: "numeric arguments in parentheses that following the type name (ex: "VARCHAR(255)") are ignored by SQLite". Testado no livewire-starter-kit (Laravel 13).

ALÉM DA AULA · TIRE O GRUPO NO ARQUIVO, NÃO NA LINHA DE COMANDO

O juiz é o comando que o loop detecta (aula 4.1), e ele não sabe das suas flags. Ponha a exclusão no arquivo: `<groups><exclude>` no `phpunit.xml`, `addopts = "-m 'not integration'"` no `pyproject.toml`. A linha de comando vence o arquivo: `--group=integration` roda o smoke mesmo com o exclude, e `-m integration` vence o `addopts`. No pytest, o import do SDK fica dentro do teste: o arquivo é importado mesmo com o marcador fora, e um import que falha derruba a coleta inteira.

Pest, "Grouping Tests" e "Filtering Tests"; pytest, "Working with custom markers". Testado no Pest 5.3.5 e no pytest 9.2.0.

ALÉM DA AULA · A MESMA LINHA QUEBRA NUMA VERSÃO E NÃO EM OUTRA

O PR #530 do `laravel/ai`, "Make OpenAI strict mode opt-in via Strict attribute", saiu na 0.7.0, em 19/05/2026, quatro dias depois do Lab: daí em diante o strict só vale com o atributo `Strict` no agente. A armadilha muda de lugar; o teste do caminho real continua. O `SchemaEstritoTest.php` força `strict: true` de propósito: confere a regra mais dura. Campo opcional, no strict, continua no `required`: "it is possible to emulate an optional parameter by using a union type with null" (OpenAI, Structured Outputs).

github.com/laravel/ai, PR #530 e issue #506: "nothing in the docs mentions that `->required()` is needed on every parameter."

ALÉM DA AULA · PROTEJA DEPOIS DE REVISAR, E PEÇA TAMBÉM

A guarda só deixa o agente tirar o marcador de pendente (`->todo()`, `.skip`, `@pytest.mark.skip`) dos testes do `aceite.txt`. Criar o arquivo pela primeira vez passa; o resto volta com exit 2 e o motivo. Liste o teste **depois** de revisado e visto vermelho: listado antes, o agente não consegue corrigir o teste que acabou de escrever. O limite é o da doc, "Rules match tool calls, not intent": Bash e NotebookEdit passam por fora. Junto da trava, para o CLAUDE.md: "Tests are there to verify correctness, not to define the solution."

Hook testado com 16 casos de JSON falso. Claude Code, Best practices, "Limit access with permission rules and the sandbox", e Tools reference; Anthropic, Prompting best practices, "Avoid focusing on passing tests and hardcoding".

ALÉM DA AULA · A NAVEGAÇÃO QUE PEGOU O 400

O 400 do caso não saiu de teste nenhum. Ao vivo, o Lucas criou um usuário, fez login, pediu uma análise e acompanhou `tail -f storage/logs/laravel.log`. Ali apareceram, em ordem, um `TypeError` que nem deixava o pedido sair, o `RequestException` com o 400 e, depois da correção, o `SQLSTATE[22001]` da coluna curta. Repita no fim da feature, antes do PR, com a configuração de staging (chave real, banco igual ao de produção): o caminho principal com dados reais e o log aberto, atrás de `TypeError`, `RequestException` e `SQLSTATE`. Com o pacote `laravel/pail`, `php artisan pail` segue qualquer driver de log.

laravel.log do caso, 15/05: 10:22 (TypeError), 10:34 (400) e 11:10 (SQLSTATE). Laravel, Logging, "Tailing Log Messages Using Pail".

3 Instale e faça *no seu repositório*

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale a guarda do teste no seu projeto.

Cria `.claude/hooks/guarda-aceite.py` e `.claude/aceite.exemplo.txt` e soma o hook ao seu `.claude/settings.json` (veja no fim da seção). Precisa de `python3` e `jq`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 4.2
```

3 Teste o hook antes de confiar nele.

O método da aula 2.4: JSON falso no stdin, numa pasta temporária, sem tocar no seu projeto. O esperado é "16 passaram, 0 falharam".

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.2-o-teste-verde-que-mente
bash $A/modelos/testes/testa-guarda.sh .claude/hooks/guarda-aceite.py
```

Na sessão, `/hooks` mostra o hook em PreToolUse, vindo de project settings.

4 Rode o contrato do caso, em qualquer stack.

Só precisa de Python 3. O schema do caso (reconstrução) dá 6 campos fora do `required` e `exit 1`; o corrigido dá "ok". Ele também lê o corpo do pedido (Responses API ou Chat Completions): capture o que o seu SDK manda e passe por ele.

Terminal · em qualquer pasta — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.2-o-teste-verde-que-mente
M=$A/modelos; F=$M/fixtures
python3 -I $M/schema_estrito.py $F/schema-antes.json || echo "exit $?"
python3 -I $M/schema_estrito.py $F/schema-depois.json
```

5 Na sua feature-alvo: ache o ponto de fora e peça o contrato.

Abra o Claude Code na raiz do projeto. Fora é API, fila, e-mail, arquivo, coluna com limite e o próprio banco. Escolha o ponto mais arriscado:

Dentro do Claude Code — sessão

```
> onde a minha feature-alvo fala com fora?
> escreva o teste de contrato do ponto mais arriscado
```

Antes de rodar, leia: monta o que sai pelo caminho do SDK, ou escreveu o JSON à mão (que prova só o JSON escrito)? Sem teste nenhum no projeto, este é o primeiro. Pedidos em `modelos/pedir-o-contrato.md`.

6 Se o ponto é structured output com laravel/ai: o teste da aula.

Copie o contrato do schema e troque o agente do caso (`AnalisaProduto`) pelos agentes do seu app.

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.2-o-teste-verde-que-mente
mkdir -p tests/Feature/Ai
cp $A/modelos/laravel/SchemaEstritoTest.php tests/Feature/Ai/
```

Dentro do Claude Code — sessão

```
> troque AnalisaProduto no SchemaEstritoTest pelos agentes do app
```

7 Veja vermelho uma vez, pelo motivo certo.

Tire um `->required()` do seu agente e rode: o vermelho traz o nome do campo (`product.name`, no caso). Desfaça: verde. Com Sail, `vendor/bin/sail` no lugar de `php artisan`.

Terminal · projeto Laravel — laravel

```
php artisan test --filter=SchemaEstrito
```

8 Ponha o teste sob guarda (depois de revisado).

Um caminho por linha, relativo à raiz, com curinga se quiser (`tests/Feature/Ai/*Test.php`).

Terminal · na raiz do seu projeto — shell

```
echo 'tests/Feature/Ai/SchemaEstritoTest.php' >> .claude/aceite.txt
cat .claude/aceite.txt
```

9 Prove que a guarda pega o seu teste.

Uma edição que afrouxa o assert, igual à que o agente mandaria. O esperado é "Bloqueado: essa edição muda o teste, não só o marcador" e `exit 2`.

Terminal · na raiz do seu projeto — shell

```
T=tests/Feature/Ai/SchemaEstritoTest.php
j='{"tool_name":"Edit","tool_input":{"file_path":"'$T'",'
j+='"old_string":"toBe([])","new_string":"toBeArray()"}}'
echo "$j" | python3 .claude/hooks/guarda-aceite.py || echo "exit $?"
```

10 A chamada real, fora do juiz (Laravel).

O smoke grava a resposta crua em `tests/Fixtures/openai/gravada/`; o dublê serve essa gravada e a borda que ele monta sobre ela (`respostaDeBorda()`). Troque o agente nos dois.

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.2-o-teste-verde-que-mente
L=$A/modelos/laravel
mkdir -p tests/Fixtures/openai/gravada
cp $L/SmokeOpenAiTest.php $L/DubleNaBordaHttpTest.php tests/Feature/Ai/
cp $L/gravada/analisa-produto.json tests/Fixtures/openai/gravada/
```

Cole o bloco de `phpunit-groups.xml` no `phpunit.xml`, depois de `</testsuites>`. O primeiro comando é o juiz; o segundo, só o smoke, com a `OPENAI_API_KEY` do `.env` (sem ela, pula). Depois, olhe o `git diff` da gravada.

Terminal · projeto Laravel — laravel

```
php artisan test
php artisan test --group=integration
```

11 A chamada real, fora do juiz (Python).

Junte o bloco de `pyproject-integration.toml` ao seu `pyproject.toml` e troque o import dentro do teste. Ele grava a forma do que a sua função devolveu; o docstring diz como montar o dublê.

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.2-o-teste-verde-que-mente
cp $A/modelos/python/test_smoke_openai.py tests/
```

Terminal · projeto Python — pytest

```
pytest -q
pytest -m integration
```

12 Leve o ponto de fora para a spec, antes da 4.3.

Cole o bloco "Pontos de fora" de `modelos/integracao-na-spec.md` no `FEATURE-ALVO.md`: critérios binários no formato da 4.3 (smoke aceito, contrato pelo SDK, dublê com a gravada e a borda, coluna que comporta o maior texto, erro tratado). Preencha com o que voltar de:

Dentro do Claude Code — sessão

```
> antes do plano, rode um smoke real do ponto de fora e liste os campos
```

O `/plan` da 4.3 lê o `FEATURE-ALVO.md` inteiro e extrai os critérios: confirme os do bloco no primeiro checkpoint.

A guarda *instalada*

O `settings.json` que o `instalar.sh` soma ao seu. O hook roda em toda edição do Claude, mas só age nos arquivos do `aceite.txt`; o código, comentado, está em `projeto/.claude/hooks/`. O `MultiEdit` do matcher vem do slide: não está mais na lista de ferramentas da doc (Tools reference) e fica ali sem custo.

`.claude/settings.json` — instalado pelo instalar.sh · não copie do PDF

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          { "type": "command", "timeout": 10,
            "command": "python3 \"$CLAUDE_PROJECT_DIR\"/.claude/hooks/guarda-aceite.py" }
        ]
      }
    ]
  }
}
```

4 Arquivos que *reforçam a aula*

`.claude/hooks/guarda-aceite.py`

O juiz fora do alcance do réu. O mesmo contrato do hook da 2.3 (exit 2 e o motivo no stderr), com outro alvo: os testes do `aceite.txt`. Libera só tirar o marcador de pendente, que é o passo da 4.3.

No repositório: modulo-4-prova/aula-4.2-o-teste-verde-que-mente/projeto/

`modelos/schema_estrito.py + fixtures/`

O contrato do caso para qualquer stack: lê o JSON Schema ou o corpo do pedido, anda em todo nível (objeto, `items`, `anyOf`) e lista cada caminho fora do `required`. Só biblioteca padrão.

No repositório: modulo-4-prova/aula-4.2-o-teste-verde-que-mente/modelos/

`modelos/laravel/SchemaEstritoTest.php`

O teste que faltou no Worthly: monta o schema pelo mesmo `ObjectSchema` que o SDK usa antes de enviar. Vermelho com os 6 campos no agente `.antes.php`, verde no `.depois.php`.

No repositório: modulo-4-prova/aula-4.2-o-teste-verde-que-mente/modelos/laravel/

`modelos/laravel/DubleNaBordaHttpTest.php`

O dublê que respeita o provedor: troca só a rede e serve a gravada (a forma real), a borda montada sobre ela (texto longo, `null`) e o 400 real. Pegou o `TypeError` que o fake do SDK deixou passar.

No repositório: modulo-4-prova/aula-4.2-o-teste-verde-que-mente/modelos/laravel/ (com gravada/)

`modelos/laravel/SmokeOpenAiTest.php + phpunit-groups.xml`

A chamada real separada do juiz: grupo `integration`, pulada sem chave. Grava só a gravada, nunca o pior caso. O equivalente em pytest está em `modelos/python/`.

No repositório: modulo-4-prova/aula-4.2-o-teste-verde-que-mente/modelos/laravel/ e modelos/python/

`modelos/integracao-na-spec.md`

O ponto de fora na especificação: o bloco "Pontos de fora" para o `FEATURE-ALVO.md`, com critérios binários no formato da 4.3, o pedido do smoke e o caso da aula preenchido.

No repositório: modulo-4-prova/aula-4.2-o-teste-verde-que-mente/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`Target class ... does not exist` no `SchemaEstritoTest`	O dataset ainda aponta para o agente do caso	Troque pelos agentes do seu app (passo 6)
O smoke roda dentro do loop e fica vermelho ou pulado	O grupo saiu só com `--exclude-group` na sua linha de comando	Exclua no `phpunit.xml` ou no `addopts` do pyproject
pytest: "Interrupted: 1 error during collection"	Import do SDK no topo do arquivo do smoke	Import dentro do teste, como no modelo
"Bloqueado: essa edição muda o teste, não só o marcador", e o agente não corrige o teste que escreveu	O teste entrou no `aceite.txt` antes da revisão	Tire a linha, revise, veja vermelho e liste de novo
A guarda não bloqueia nada	Falta `.claude/aceite.txt`, ou o caminho não é relativo à raiz	Passo 8 com caminho relativo; `/hooks`; passo 9
Teste de gravação verde e produção com "value too long"	SQLite em memória ignora o tamanho do `VARCHAR`	Juiz no mesmo banco de produção, `max:N` na validação ou `text`
O teste que o Claude roda volta em JSON (`{"tool":"pest",...}`)	O Pest 5 percebe a sessão de agente e troca a saída	É normal. No seu terminal, a saída é a de sempre

6 Você sai *com*

- [ ] `testa-guarda.sh` no seu projeto: "16 passaram, 0 falharam".
- [ ] `schema_estrito.py`: 6 campos e `exit 1` no schema do caso; "ok" no corrigido.
- [ ] O contrato do ponto mais arriscado da sua feature-alvo, visto vermelho com o nome do campo.
- [ ] O smoke fora do juiz (`--group=integration`) e o dublê verde com a gravada e a borda.
- [ ] O contrato no `.claude/aceite.txt`, e a guarda dando `exit 2` para um assert afrouxado.
- [ ] O bloco "Pontos de fora" no `FEATURE-ALVO.md`, preenchido com os campos do smoke.
- [ ] Antes do PR: naveguei a feature com a chave real e o log aberto.
- [ ] `.claude/` e os testes no commit: o time e o loop recebem a mesma trava.

PROMETIDO NA AULA

- ✓ **"O Pest monta o esquema pelo caminho do SDK"**: `SchemaEstritoTest.php`, passos 6 e 7; `schema_estrito.py`, passo 4.
- ✓ **"Chamar a API de verdade"**: `SmokeOpenAiTest.php` e `test_smoke_openai.py`, passos 10 e 11 (p. 7).
- ✓ **"Dublê que respeita o provedor"**: `DubleNaBordaHttpTest.php` (gravada, borda, 400), passo 10.
- ✓ **"Enquanto especifica, um smoke real"**: a gravada (passo 10) e o pedido do passo 12 (p. 7).
- ✓ **"Grupo separado"** (slide): `phpunit-groups.xml` e `pyproject-integration.toml`, passos 10 e 11 (p. 7).
- ✓ **"O processo de especificação"** e **"esses agentes também vão mexer nesses testes"**: os critérios de `integracao-na-spec.md`, que o `/plan` da 4.3 leva à SPEC, passo 12 (p. 7).
- ✓ **"O agente também mexe no teste"** (slide): `guarda-aceite.py`, `instalar.sh 4.2`, passos 2, 3, 8 e 9.
- ✓ **"Automatizar essa navegação com Playwright"**: aula 4.4, em `modulo-4-prova/aula-4.4-e2e-minimo/`; à mão, seção 2 (p. 4).
- ✓ **"Vamos construir a nossa especificação"**: aula 4.3, em `modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/`, com o bloco do passo 12.

7 *Glossário*

Dublê (mock)	O que o teste põe no lugar de uma dependência real: interface, SDK ou rede.
Teste de contrato	Confere o que sai (ou volta) contra a regra do outro lado, sem chamá-lo.
Structured output	Resposta presa a um JSON Schema; no strict, toda chave vai no `required`.
nullable, required	`nullable`: o valor pode vir `null`. `required`: a chave tem que existir.
Smoke real	Uma chamada com chave de verdade, num grupo fora do juiz.
Gravada, borda	A resposta crua que o smoke grava; e ela com o pior caso escrito à mão.
Juiz	O comando que decide se está pronto, fora da sessão (aula 4.1). Sem rede.

8 Para *ir além*

- OpenAI · Structured Outputs: "All fields must be required" e o opcional como união com `null`.
- laravel/ai · PR #530 e issue #506: o strict opcional e o `->required()`.
- Laravel · HTTP Client, seção Testing: `Http::fake()`, `Http::assertSent()` e `preventStrayRequests()`.
- Pest · Grouping Tests e pytest · Working with custom markers: grupo, marcador e seleção.
- Claude Code · Hooks reference: o exit 2 no `PreToolUse` e o menu `/hooks`.

Próxima: *Aula 4.3 · Spec com critério de aceite verificável*

Então qual teste escrever? O que o critério pedir: o FEATURE-ALVO.md vira critério binário, e cada critério ganha um teste.